42.2 核心功能实现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~355k tokens，输出 ~6k tokens（本节）

本节实现 MiniClaw 的核心骨架：WebSocket Gateway、消息路由、Agent 运行循环和基础工具系统。

42.2.1 搭建 WebSocket Gateway 骨架

Gateway 是 MiniClaw 的中枢，负责接受客户端连接并路由消息。

最小 RPC 协议

参考 OpenClaw 的帧协议（第 3 章），定义一个简化版：

```typescript
// src/gateway/protocol.ts

export type RpcRequest = {
  type: "req";
  id: string;       // 请求 ID
  method: string;    // 方法名
  params?: unknown;  // 参数
};

export type RpcResponse = {
  type: "res";
  id: string;       // 对应请求 ID
  result?: unknown;  // 成功结果
  error?: string;    // 错误信息
};

export type RpcEvent = {
  type: "event";
  event: string;     // 事件名
  payload?: unknown;  // 事件数据
};

export type RpcFrame = RpcRequest | RpcResponse | RpcEvent;
```

WebSocket 服务器

```
// src/gateway/server.ts

import { WebSocketServer, WebSocket } from "ws";
import { randomUUID } from "node:crypto";
import type { RpcFrame, RpcRequest, RpcResponse } from "./protocol.js";

type ClientConnection = {
  id: string;
  ws: WebSocket;
  subscribedEvents: Set<string>;
};

export class Gateway {
  private wss: WebSocketServer;
  private clients = new Map<string, ClientConnection>();
  private handlers = new Map<string, (params: unknown) => Promise<unknown>>();

  constructor(port: number) {
    this.wss = new WebSocketServer({ port });
    this.wss.on("connection", (ws) => this.handleConnection(ws));
  }

  // 注册 RPC 方法处理器
  registerMethod(method: string, handler: (params: unknown) => Promise<unknown>) {
    this.handlers.set(method, handler);
  }

  // 向所有订阅者广播事件
  broadcast(event: string, payload?: unknown) {
    const frame: RpcFrame = { type: "event", event, payload };
    const data = JSON.stringify(frame);
    for (const client of this.clients.values()) {
      if (client.subscribedEvents.has(event) || client.subscribedEvents.has("*")) {
        client.ws.send(data);
      }
    }
  }

  private handleConnection(ws: WebSocket) {
    const clientId = randomUUID();
    const client: ClientConnection = {
```

```
      id: clientId,
      ws,
      subscribedEvents: new Set(["*"]),  // 默认订阅所有事件
    };
    this.clients.set(clientId, client);

    ws.on("message", async (raw) => {
      try {
        const frame = JSON.parse(raw.toString()) as RpcFrame;
        if (frame.type === "req") {
          await this.handleRequest(client, frame);
        }
      } catch (err) {
        console.error("Frame parse error:", err);
      }
    });

    ws.on("close", () => {
      this.clients.delete(clientId);
    });
  }

  private async handleRequest(client: ClientConnection, req: RpcRequest) {
    const handler = this.handlers.get(req.method);
    const res: RpcResponse = { type: "res", id: req.id };
    if (!handler) {
      res.error = `Unknown method: ${req.method}`;
    } else {
      try {
        res.result = await handler(req.params);
      } catch (err) {
        res.error = err instanceof Error ? err.message : String(err);
      }
    }
    client.ws.send(JSON.stringify(res));
  }
}
```

这个 Gateway 实现了 OpenClaw 三帧协议的核心：请求-响应 + 事件广播。

42.2.2 实现消息路由与会话管理

会话存储

```
// src/gateway/session-store.ts

export type Session = {
  key: string;
  messages: Array<{
    role: "user" | "assistant" | "system";
    content: string;
  }>;
  model: string;
  createdAt: number;
  updatedAt: number;
};

export class SessionStore {
  private sessions = new Map<string, Session>();

  getOrCreate(key: string, model: string): Session {
    let session = this.sessions.get(key);
    if (!session) {
      session = {
        key,
        messages: [],
        model,
        createdAt: Date.now(),
        updatedAt: Date.now(),
      };
      this.sessions.set(key, session);
    }
    return session;
  }

  addMessage(key: string, role: "user" | "assistant", content: string) {
    const session = this.sessions.get(key);
    if (!session) return;
    session.messages.push({ role, content });
    session.updatedAt = Date.now();
  }
```

```
  getMessages(key: string) {
    return this.sessions.get(key)?.messages ?? [];
  }

  reset(key: string) {
    this.sessions.delete(key);
  }
}
```

路由注册

在 Gateway 上注册核心 RPC 方法：

```typescript
// src/gateway/routes.ts

export function registerRoutes(
  gateway: Gateway,
  sessions: SessionStore,
  agent: AgentRunner,
) {
  // 发送消息给 Agent
  gateway.registerMethod("chat.send", async (params) => {
    const { sessionKey, message } = params as {
      sessionKey: string;
      message: string;
    };
    const session = sessions.getOrCreate(sessionKey, "claude-sonnet-4-20250514");
    sessions.addMessage(sessionKey, "user", message);

    // 异步执行 Agent 运行循环
    const reply = await agent.run({
      sessionKey,
      messages: sessions.getMessages(sessionKey),
    });

    sessions.addMessage(sessionKey, "assistant", reply);

    // 广播聊天事件
    gateway.broadcast("chat", {
      sessionKey,
      state: "final",
      message: { role: "assistant", content: reply },
    });

    return { ok: true };
  });

  // 会话管理
  gateway.registerMethod("sessions.list", async () => {
    // 返回会话列表
  });

  gateway.registerMethod("sessions.reset", async (params) => {
    const { key } = params as { key: string };
    sessions.reset(key);
    return { ok: true };
```

```
  });
}
```

42.2.3 对接 LLM API

Anthropic 适配器

```
// src/providers/anthropic.ts

import Anthropic from "@anthropic-ai/sdk";

export type ToolDefinition = {
  name: string;
  description: string;
  input_schema: Record<string, unknown>;
};

export type Message = {
  role: "user" | "assistant";
  content: string | Array<{ type: string; [key: string]: unknown }>;
};

export class AnthropicProvider {
  private client: Anthropic;

  constructor(apiKey: string) {
    this.client = new Anthropic({ apiKey });
  }

  async chat(params: {
    model: string;
    messages: Message[];
    tools?: ToolDefinition[];
    system?: string;
    maxTokens?: number;
  }) {
    const response = await this.client.messages.create({
      model: params.model,
      max_tokens: params.maxTokens ?? 4096,
      system: params.system,
      messages: params.messages,
      tools: params.tools,
    });
    return response;
```

```
  }
}
```

Agent 运行循环

参考 OpenClaw 的 `pi-agent.ts` （第 7 章），实现核心的工具调用循环：

```typescript
// src/agent/runner.ts

export class AgentRunner {
  private provider: AnthropicProvider;
  private tools: Map<string, ToolExecutor>;
  private systemPrompt: string;

  constructor(params: {
    provider: AnthropicProvider;
    tools: ToolExecutor[];
    systemPrompt: string;
  }) {
    this.provider = params.provider;
    this.tools = new Map(params.tools.map(t => [t.name, t]));
    this.systemPrompt = params.systemPrompt;
  }

  async run(params: {
    sessionKey: string;
    messages: Message[];
  }): Promise<string> {
    const toolDefs = Array.from(this.tools.values()).map(t => t.definition);
    let messages = [...params.messages];
    const MAX_TURNS = 10;  // 最大工具调用轮次

    for (let turn = 0; turn < MAX_TURNS; turn++) {
      const response = await this.provider.chat({
        model: "claude-sonnet-4-20250514",
        messages,
        tools: toolDefs,
        system: this.systemPrompt,
      });

      // 检查是否有工具调用
      const toolUseBlocks = response.content.filter(
        (block) => block.type === "tool_use"
      );

      if (toolUseBlocks.length === 0) {
        // 无工具调用 → 提取文本回复
        const textBlock = response.content.find(
          (block) => block.type === "text"
        );
```

```
        return textBlock?.text ?? "";
      }

      // 有工具调用 → 执行并继续循环
      messages.push({
        role: "assistant",
        content: response.content,
      });

      const toolResults = [];
      for (const toolUse of toolUseBlocks) {
        const executor = this.tools.get(toolUse.name);
        let result: string;
        try {
          result = await executor.execute(toolUse.input);
        } catch (err) {
          result = `Error: ${err.message}`;
        }
        toolResults.push({
          type: "tool_result",
          tool_use_id: toolUse.id,
          content: result,
        });
      }

      messages.push({
        role: "user",
        content: toolResults,
      });
    }

    return "（达到最大工具调用轮次限制）";
  }
}
```

这个循环实现了 OpenClaw Agent 运行时的核心逻辑：发送消息 → 检查工具调用 → 执行工具 → 将结果反馈给 LLM → 重复。 `MAX_TURNS = 10` 是安全阀，防止无限循环。

42.2.4 实现基础工具系统

工具接口

```
// src/agent/tools/types.ts

export type ToolExecutor = {
  name: string;
  definition: ToolDefinition;
  execute: (input: Record<string, unknown>) => Promise<string>;
};
```

Bash 工具

```ts
// src/agent/tools/bash.ts

import { execSync } from "node:child_process";

export const bashTool: ToolExecutor = {
  name: "bash",
  definition: {
    name: "bash",
    description: "Execute a bash command and return the output.",
    input_schema: {
      type: "object",
      properties: {
        command: { type: "string", description: "The command to execute" },
      },
      required: ["command"],
    },
  },
  execute: async (input) => {
    const command = input.command as string;
    // 安全检查：禁止危险命令
    const BLOCKED = ["rm -rf /", "mkfs", "dd if=/dev"];
    if (BLOCKED.some(b => command.includes(b))) {
      return "Error: This command is blocked for safety.";
    }
    try {
      const output = execSync(command, {
        timeout: 30_000,  // 30 秒超时
        maxBuffer: 1024 * 1024,  // 1MB 输出限制
        encoding: "utf-8",
      });
      return output.slice(0, 10_000);  // 截断过长输出
    } catch (err) {
      return `Error: ${err.message}`;
    }
  },
};
```

文件读取工具

```
// src/agent/tools/read.ts

import fs from "node:fs";
import path from "node:path";

export const readTool: ToolExecutor = {
  name: "read",
  definition: {
    name: "read",
    description: "Read the contents of a file.",
    input_schema: {
      type: "object",
      properties: {
        filePath: { type: "string", description: "Absolute file path" },
      },
      required: ["filePath"],
    },
  },
  execute: async (input) => {
    const filePath = input.filePath as string;
    // 安全：限制在工作目录内
    const resolved = path.resolve(filePath);
    if (!resolved.startsWith(process.cwd())) {
      return "Error: Access denied. File outside workspace.";
    }
    try {
      return fs.readFileSync(resolved, "utf-8").slice(0, 50_000);
    } catch (err) {
      return `Error: ${err.message}`;
    }
  },
};
```

入口文件

将所有组件组装在一起：

```typescript
// src/index.ts

import { Gateway } from "./gateway/server.js";
import { SessionStore } from "./gateway/session-store.js";
import { AgentRunner } from "./agent/runner.js";
import { AnthropicProvider } from "./providers/anthropic.js";
import { bashTool } from "./agent/tools/bash.js";
import { readTool } from "./agent/tools/read.js";
import { registerRoutes } from "./gateway/routes.js";

const provider = new AnthropicProvider(process.env.ANTHROPIC_API_KEY!);
const agent = new AgentRunner({
  provider,
  tools: [bashTool, readTool],
  systemPrompt: "你是一个有帮助的 AI 助手。你可以执行命令和读取文件来帮助用户。",
});

const sessions = new SessionStore();
const gateway = new Gateway(3000);
registerRoutes(gateway, sessions, agent);

console.log("MiniClaw Gateway running on ws://localhost:3000");
```

本节小结

1. **Gateway** 实现了 OpenClaw 三帧协议（req/res/event）的简化版，支持 RPC 方法注册和事件广播。
2. **会话存储** 使用内存 Map，提供创建/读取/重置等基本操作。
3. **LLM 适配器** 封装 Anthropic SDK，统一 API 接口。
4. **Agent 运行循环** 实现了核心的"调用 LLM → 检查工具调用 → 执行工具 → 反馈结果"循环，最大 10 轮。
5. **工具系统** 以统一接口定义，Bash 工具包含超时（30s）和危险命令拦截，文件读取工具限制在工作目录内。